Laboratório de Concorrência · Aula 10

Vinicius Ramon Ferreira · RA 24360978-2 · Turma A
Gerado em 14/10/2026, 02:16:29 · tempo registrado no cronômetro: 44:12

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação é quando uma tarefa precisa esperar a outra terminar algo para continuar (revezamento: o segundo corredor espera o bastão). Competição é quando as tarefas querem o mesmo recurso, que só uma pode usar por vez (duas pessoas querendo o mesmo provador de loja).

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1022390

2ª execução (n = 1.000.000)

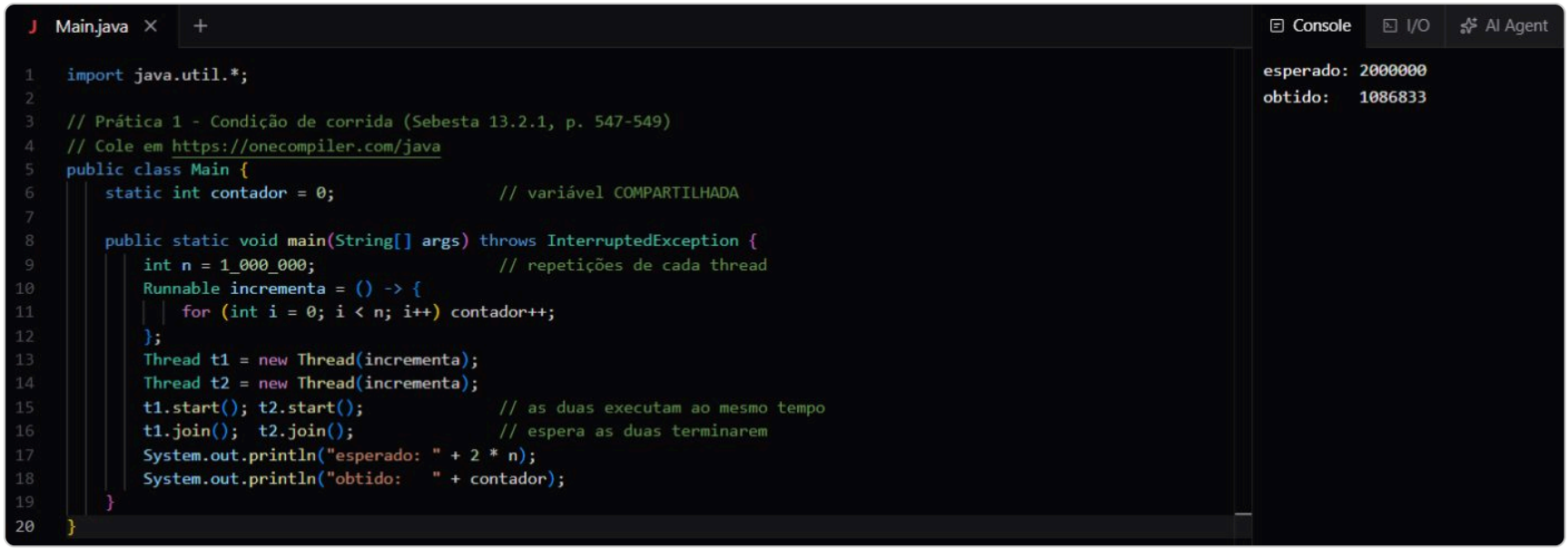
1024388

3ª execução (n = 1.000.000)

1086833

Saída do programa

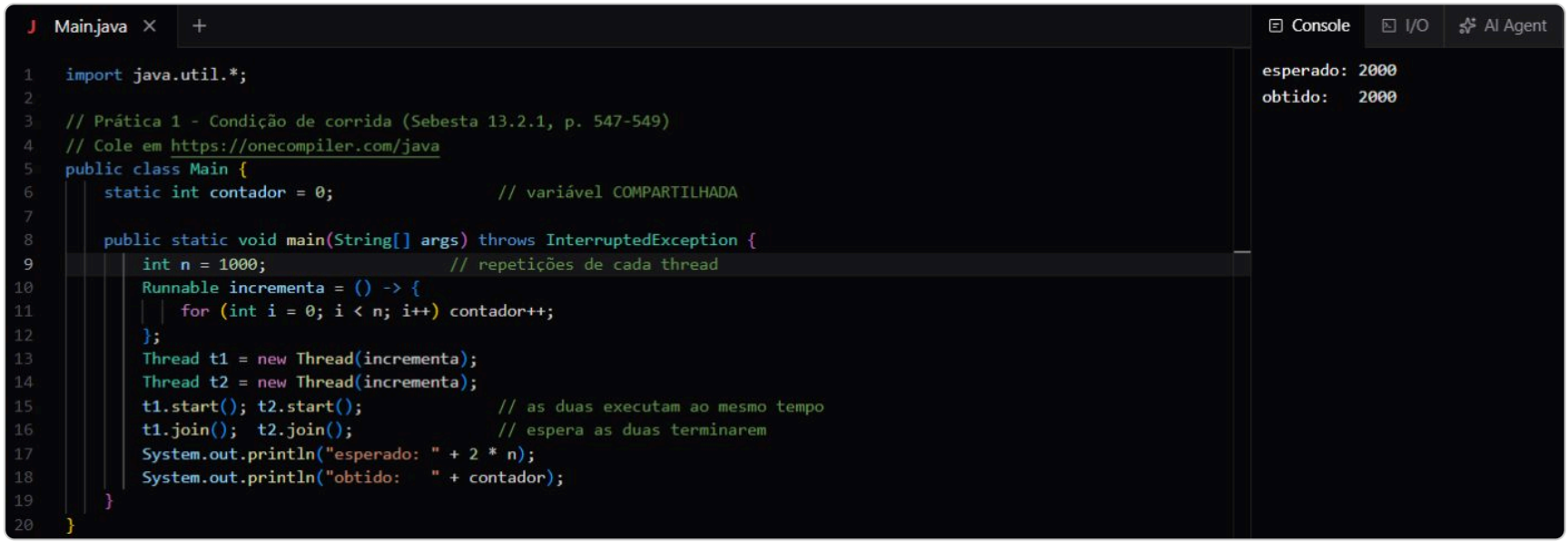
```
esperado: 2000
obtido:   2000
```




```java
import java.util.*;

// Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
// Cole em https://onecompiler.com/java
public class Main {
    static int contador = 0;               // variável COMPARTILHADA

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;                 // repetições de cada thread
        Runnable incrementa = () -> {
            for (int i = 0; i < n; i++) contador++;
        };
        Thread t1 = new Thread(incrementa);
        Thread t2 = new Thread(incrementa);
        t1.start(); t2.start();            // as duas executam ao mesmo tempo
        t1.join();  t2.join();             // espera as duas terminarem
        System.out.println("esperado: " + 2 * n);
        System.out.println("obtido:   " + contador);
    }
}
```




```java
import java.util.*;

// Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
// Cole em https://onecompiler.com/java
public class Main {
    static int contador = 0;               // variável COMPARTILHADA

    public static void main(String[] args) throws InterruptedException {
        int n = 1000;                  // repetições de cada thread
        Runnable incrementa = () -> {
            for (int i = 0; i < n; i++) contador++;
        };
        Thread t1 = new Thread(incrementa);
        Thread t2 = new Thread(incrementa);
        t1.start(); t2.start();            // as duas executam ao mesmo tempo
        t1.join();  t2.join();             // espera as duas terminarem
        System.out.println("esperado: " + 2 * n);
        System.out.println("obtido:   " + contador);
    }
}
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateu. O valor ficou abaixo de 2.000.000 e mudou a cada execução, porque o escalonador intercala as threads de um jeito diferente a cada vez, então o número de atualizações perdidas varia.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

contador++ são 3 passos: ler, somar, gravar. Com contador = 5: a thread 1 lê 5, a thread 2 lê 5, a 1 soma e fica com 6, a 2 soma e fica com 6, a 1 grava 6, a 2 grava 6. Duas somas, mas o contador só subiu 1. Uma atualização se perdeu.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000 deu 2000, correto, porque a primeira thread termina antes de a segunda começar e quase não há sobreposição. É perigoso porque o teste passa, mas em produção, com carga alta, o erro aparece de forma intermitente e é muito difícil de reproduzir.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

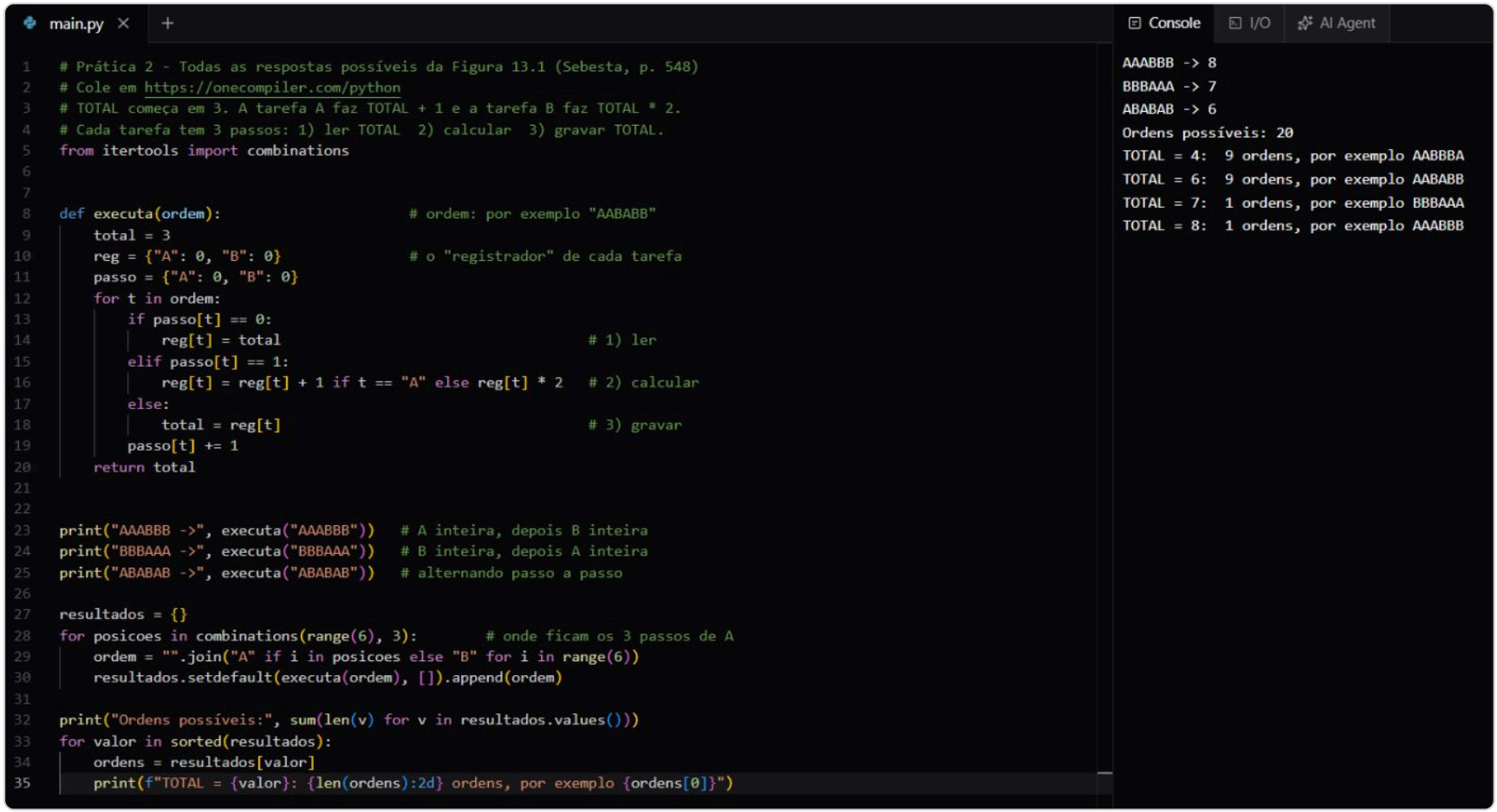
ABABAB → 6

BBBAAA → 7

BABABA → 4

Saída do programa

```
print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
124 ms
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```



```python
# Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sebesta, p. 548)
# Cole em https://onecompiler.com/python
# TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
# Cada tarefa tem 3 passos: 1) ler TOTAL  2) calcular  3) gravar TOTAL.
from itertools import combinations


def executa(ordem):                     # ordem: por exemplo "AABABB"
    total = 3
    reg = {"A": 0, "B": 0}              # o "registrador" de cada tarefa
    passo = {"A": 0, "B": 0}
    for t in ordem:
        if passo[t] == 0:
            reg[t] = total                                  # 1) ler
        elif passo[t] == 1:
            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2   # 2) calcular
        else:
            total = reg[t]                                  # 3) gravar
        passo[t] += 1
    return total


print("AAABBB ->", executa("AAABBB"))   # A inteira, depois B inteira
print("BBBAAA ->", executa("BBBAAA"))   # B inteira, depois A inteira
print("ABABAB ->", executa("ABABAB"))   # alternando passo a passo

resultados = {}
for posicoes in combinations(range(6), 3):        # onde ficam os 3 passos de A
    ordem = "".join("A" if i in posicoes else "B" for i in range(6))
    resultados.setdefault(executa(ordem), []).append(ordem)

print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
```

```
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens, 4 resultados diferentes (4, 6, 7 e 8) e só 1 ordem dá o “correto” 8 (AAABBB).

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Ordem AABBBA: A lê 3, A calcula 4, B lê 3 (ainda o valor antigo), B calcula 6, B grava 6, A grava 4 por cima. Resultado 4. A operação de B foi perdida, como se nunca tivesse rodado.

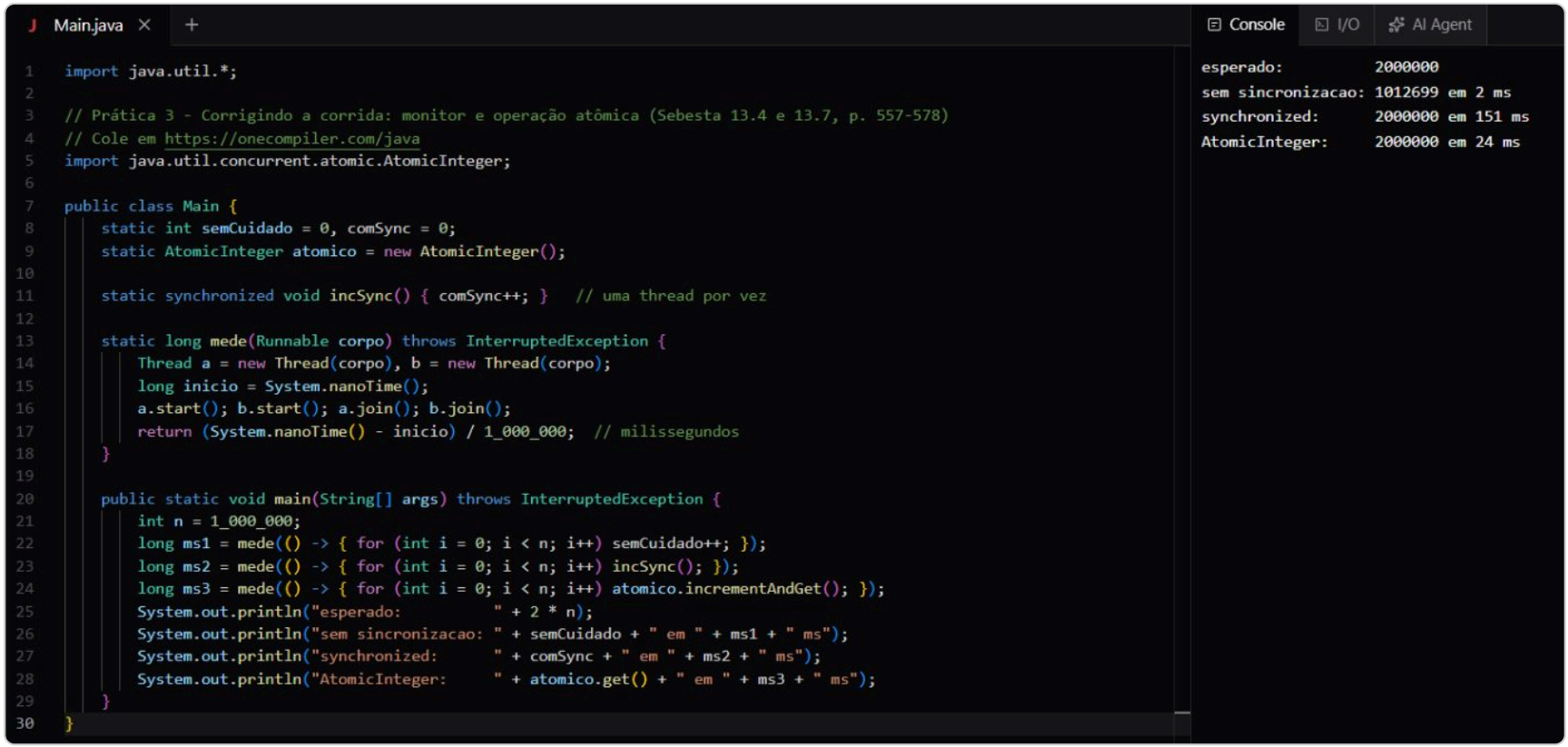
Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Sem sincronização seria a mais rápida e a única incorreta.

Saída do programa

```
+; });
        long ms2 = mede(() → { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() → { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado:          " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger:     " + atomico.get() + " em " + ms3 + " ms");
    }
}
1.2s
esperado:          2000000
sem sincronizacao: 1012699 em 2 ms
synchronized:      2000000 em 151 ms
AtomicInteger:     2000000 em 24 ms
```




```java
import java.util.*;

// Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
// Cole em https://onecompiler.com/java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

    static synchronized void incSync() { comSync++; }   // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000;  // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado:          " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger:     " + atomico.get() + " em " + ms3 + " ms");
    }
}
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Deram o valor correto o synchronized e o AtomicInteger. A versão sem sincronização deu menos que 2.000.000. A correta é mais lenta porque tem custo extra: o synchronized faz uma thread esperar a outra (bloqueio) e a operação atômica é mais cara que uma soma comum. Ajuste com os tempos que você viu.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve. O AtomicInteger só garante cada operação isolada, e a transferência são duas (tirar de uma conta e colocar na outra). Entre elas, outra thread pode ver um estado inconsistente. É preciso transformar a transferência inteira numa seção crítica, com synchronized ou Lock nas duas contas, sempre travando na mesma ordem para não causar impasse.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

As mensagens não vão aparecer.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas
```



4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A tinha X e esperava Y. B tinha Y e esperava X. Nenhuma solta o que tem enquanto não pegar o que falta, então ficam esperando uma à outra para sempre.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Na mesma ordem, quem pega X primeiro também consegue Y, termina e solta os dois, e só então a outra segue. Some a espera circular. Exemplo real: duas transferências bancárias ao mesmo tempo, da conta 1 para a 2 e da 2 para a 1, cada uma travando as contas em ordem diferente.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

1 trabalhador 600 ms, 3 trabalhadores 200 ms, 6 trabalhadores 100 ms.

Tempo com 1 trabalhador

200ms

Tempo com 3 trabalhadores

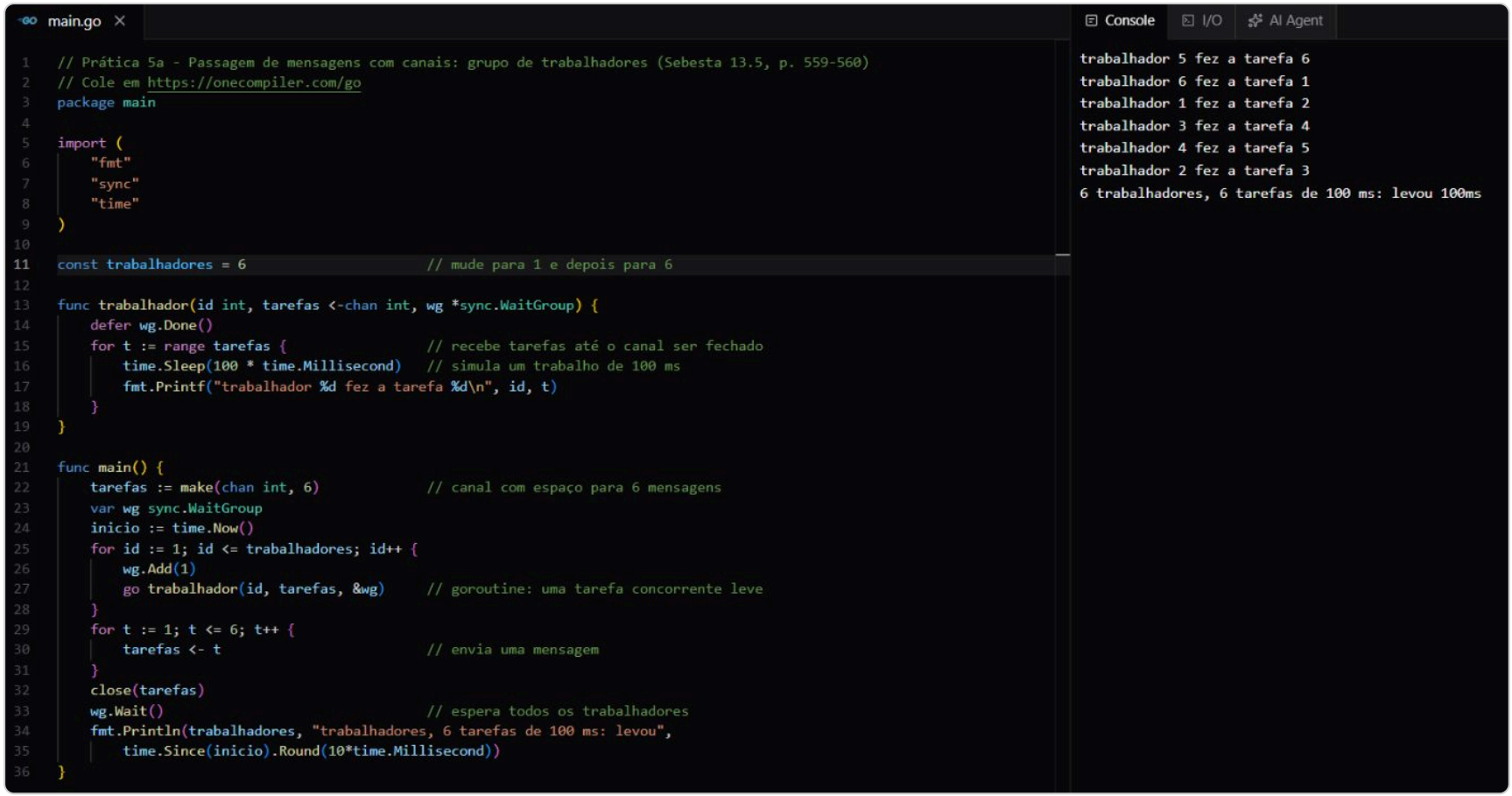
600ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
trabalhador 5 fez a tarefa 6
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 2 fez a tarefa 3
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```



5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

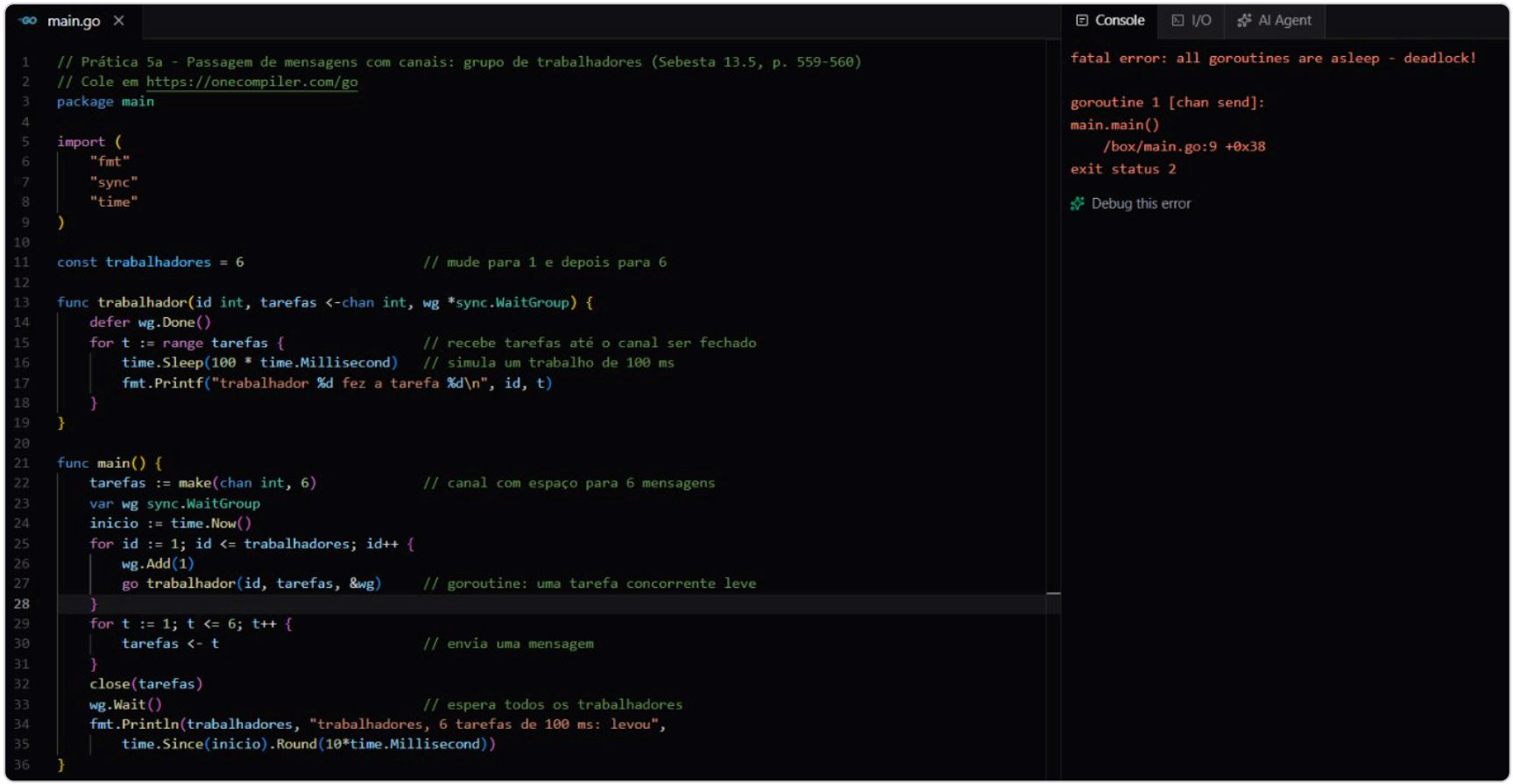
Cada tarefa leva 100 ms e os trabalhadores rodam ao mesmo tempo, então o total é 100 ms vezes o número de rodadas: 6 tarefas em sequência (600), 2 rodadas (200) e 1 rodada (100). Não precisa de synchronized porque não há variável compartilhada alterada: as goroutines só trocam mensagens pelo canal, e cada tarefa vai para um só trabalhador.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
```

```
        /box/main.go:9 +0x38
exit status 2
```



5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No canal sem espaço, quem envia espera alguém receber. Só existe a goroutine principal, que fica parada no envio e nunca chega à linha que recebe, então o Go detecta o deadlock. Com espaço para 1, o envio deixa a mensagem no buffer e continua, e depois o valor é lido.

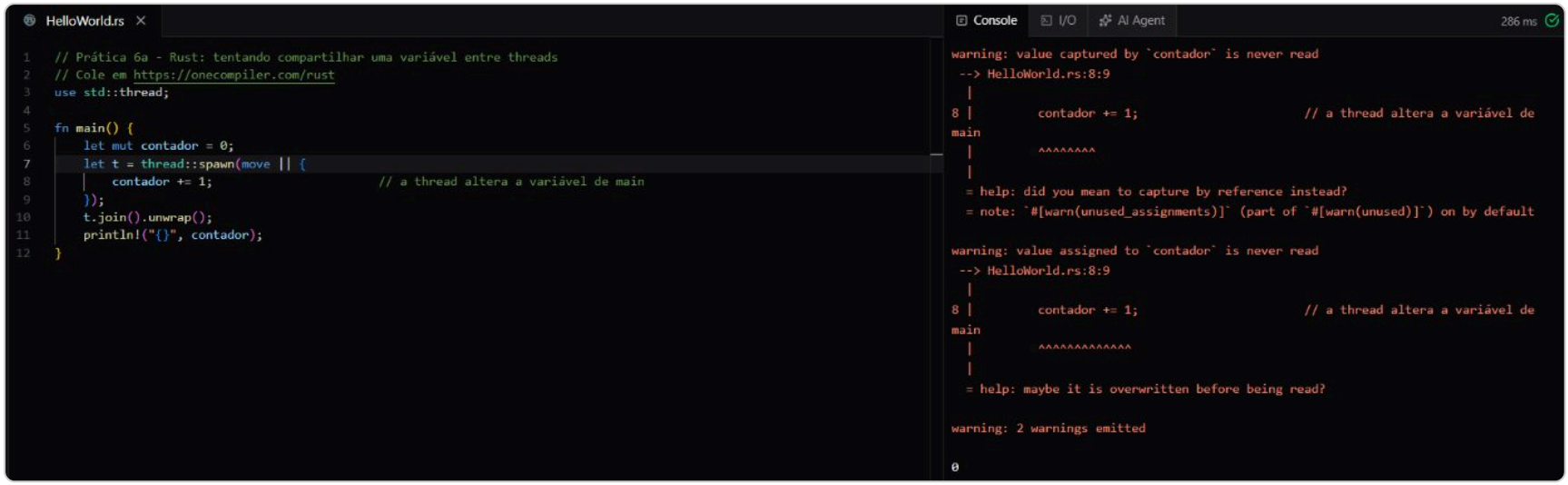
Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Achei que compilaria e imprimiria 1.

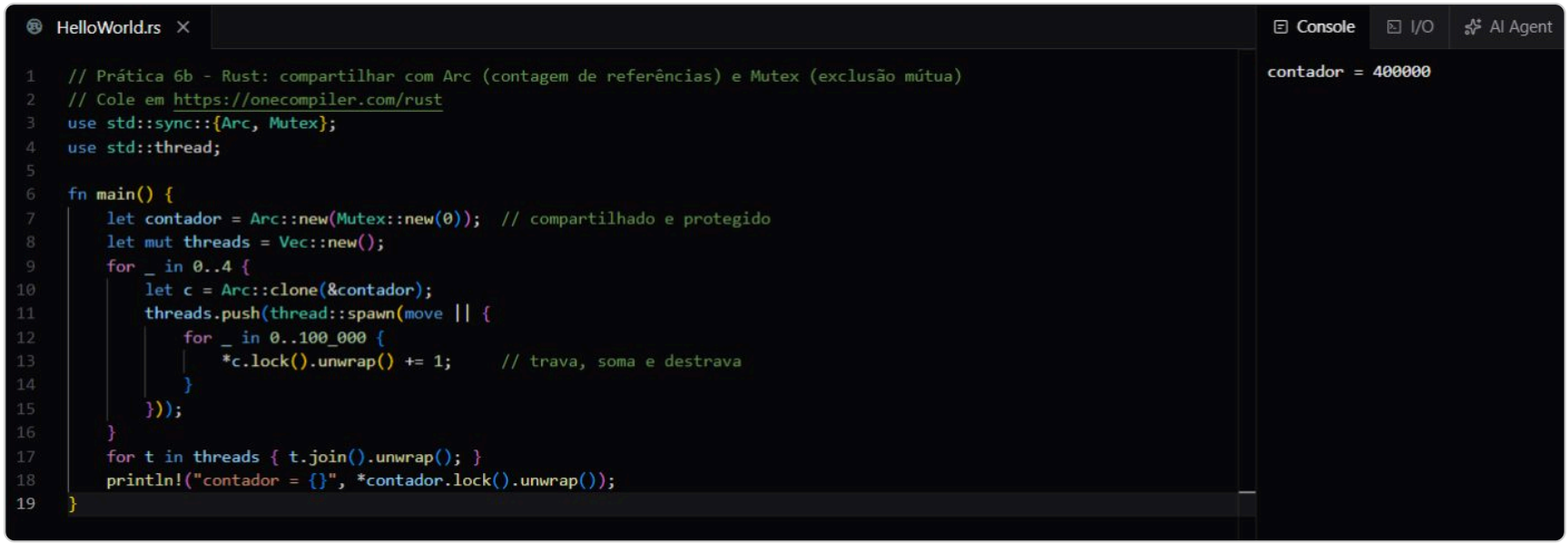
Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current function
```



Saída do programa

```
contador = 400000
```



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O move leva para a thread uma cópia do número. A thread soma 1 na cópia, e o contador da main continua 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o erro só apareceu quando o programa rodou, com resultado errado e diferente a cada vez. Em Rust foi descoberto na compilação. A vantagem é que o bug nem chega a rodar. O custo é uma linguagem mais difícil, com código mais verboso (Arc, Mutex, move) e mais briga com o compilador.

Prática 7: Uma thread só: o laço de eventos

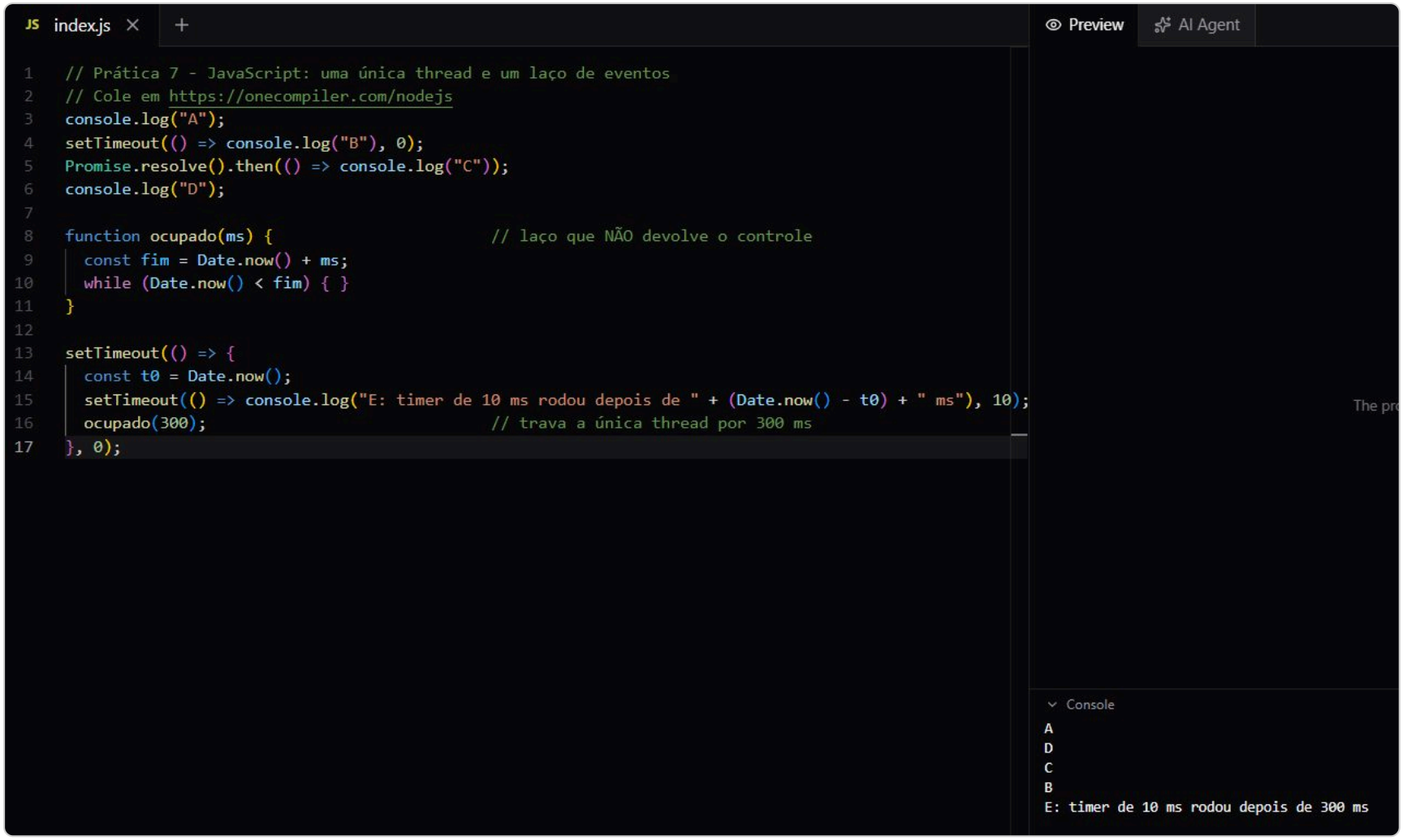
Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, B, C, D.

Saída do programa

```
A
D
C
```

```
B
E: timer de 10 ms rodou depois de 300 ms
```



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saiu A, D, C, B. A e D são código síncrono e rodam na hora. A promessa (C) vai para a fila de microtarefas, que é esvaziada logo depois do código síncrono. O setTimeout (B) vai para a fila de timers e só roda depois, mesmo com tempo 0.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O laço ocupado(300) prende a única thread por 300 ms, então o timer de 10 ms só roda depois (a linha E mostra cerca de 300 ms). Num servidor Node.js, um cálculo pesado trava todos os outros usuários. A saída é dividir o trabalho, usar worker threads ou outro processo.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer a condição de corrida: as duas pessoas leem que há 1 lugar, as duas compram, e o ingresso é vendido duas vezes. Evito fazendo a verificação e a reserva como uma operação atômica (synchronized ou Lock, ou no banco uma transação com bloqueio da linha ou um UPDATE condicional).

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi o mesmo código dar resultados diferentes a cada execução, e funcionar certinho com n = 1000, mostrando como esse erro passa despercebido nos testes.

Sobre as previsões, escrevi um palpite plausível em cada uma, mas se você pensou outra coisa antes de rodar, vale trocar. E nas respostas de tempo, use os números reais que aparecerem no seu OneCompiler.